

Université Mohammed Premier
École Nationale des Sciences Appliquées d'Oujda
Filière Génie Informatique – GI3

Rapport de Mini-Projet
Conception et Développement d'une Application de
Suivi de Traitements Médicaux-CareDash-

Réalisé par : Chaimae El maachi
Boughida Najoua

Encadrante : Mme Douae EL HILA

Module : Développement Java IHM
Année : 2025/2026

Remerciements

À l'issue de ce mini-projet de Développement Java IHM, nous tenons à exprimer notre profonde gratitude et nos sincères remerciements à notre encadrante, **Mme Douae EL HILA**, pour sa grande disponibilité, la pertinence de ses orientations pédagogiques et le précieux soutien qu'elle nous a apporté tout au long de la réalisation de ce travail.

Nous tenons également à remercier chaleureusement l'ensemble du corps professoral et administratif de l'École Nationale des Sciences Appliquées d'Oujda (ENSAO), en particulier les enseignants de la filière Génie Informatique, pour la qualité de la formation et l'environnement d'apprentissage stimulant qu'ils nous offrent au quotidien.

Enfin, nous tenons à exprimer notre vive reconnaissance à nos familles, à nos proches ainsi qu'à nos camarades de promotion pour leur soutien moral constant et leurs encouragements tout au long de notre parcours académique.

Table des matières

Remerciements	1
Introduction	6
1 Présentation du projet	7
1.1 Description de l'application	7
1.2 Besoins fonctionnels	7
1.3 Besoins non fonctionnels	9
2 Conception	10
2.1 Architecture MVC	10
2.2 Diagramme de classes UML	10
2.3 Diagramme de cas d'utilisation	12
2.4 Diagramme de séquence	12
3 Environnement de travail	14
3.1 Outils et technologies utilisés	14
3.2 Structure du projet	14
4 Répartition des tâches	17
4.1 Tâches de l'étudiant 1 – Chaimae El maachi	18
4.2 Tâches de l'étudiant 2 – Boughida Najoua	19
5 Explication du code	20
5.1 Connexion à la base de données – <code>DatabaseManager.java</code>	20
5.2 Les classes modèles	21
5.3 Les contrôleurs	21
5.4 Fonctionnalités spécifiques	25
6 Explication du code	27
6.1 Connexion à la base de données – <code>DatabaseManager.java</code>	27
6.2 Les classes modèles	28
6.3 Les contrôleurs	28
6.4 Les fichiers FXML	32
6.5 Fonctionnalités spécifiques	34
7 Interfaces de l'application	36
7.1 Fenêtre principale	36
7.2 Interfaces de gestion de l'entité 1 – Patients	38
7.3 Interfaces de gestion de l'entité 2 – Ordonnances	39

7.4	Tableau de bord – Statistiques	40
7.5	Menus et dialogues	41
7.6	Export de données	41
Conclusion Générale		43
Références		44

Table des figures

2.2.1 Diagramme de classes	11
2.3.1 Diagramme de cas d'utilisation de CareDash	12
2.4.1 Diagramme de séquence Authentification d'un utilisateur	13
3.2.1 Organisation arborescente des packages et fichiers du projet CareDash . . .	15
7.1.1 Écran de connexion	37
7.1.2 Écran d'inscription	37
7.1.3 Fenêtre principale de l'application (espace de travail)	38
7.2.1 Interface de gestion des patients	39
7.3.1 Interface de gestion des ordonnances	40
7.4.1 Tableau de bord statistique	41
7.6.1 Impression	42
7.6.2 Fiche d'ordonnance	42

Liste des tableaux

4.1	Tâches réalisées par l'étudiant 1 – Chaimae El maachi	18
4.2	Tâches réalisées par l'étudiant 2 – Boughida Najoua	19

Introduction

Dans le milieu médical, l'accès rapide, centralisé et sécurisé aux informations des patients constitue un facteur critique pour la qualité des soins. Les fiches d'ordonnances manuscrites ou l'absence d'outils analytiques modernes augmentent le risque d'erreurs cliniques et ralentissent le traitement administratif.

C'est dans ce cadre que s'inscrit notre projet : la conception et le développement d'une application de bureau intuitive et moderne pour la gestion clinique, baptisée **CareDash**. L'accent a été mis sur l'interactivité graphique grâce à **JavaFX** et l'accès dynamique à une base de données relationnelle centralisée, offrant au corps médical un environnement de travail numérisé et fluide.

Objectifs et Fonctionnalités Clés Pour répondre efficacement à ces défis, le développement de l'application s'articule autour de trois axes principaux :

- **Sécurisation et Centralisation** : Remplacement des supports manuscrits par des modules de prescription automatisés et un archivage numérique des dossiers patients pour éliminer les risques de perte.
- **Performance Technique** : Modélisation d'une base de données relationnelle optimisée, permettant des recherches, des filtres et des mises à jour d'informations en temps réel.
- **Ergonomie Médicale** : Conception d'une interface graphique réactive sous JavaFX, pensée pour réduire les manipulations administratives et optimiser le temps des soignants.

Structure du Rapport Pour présenter notre démarche de manière fluide, ce document est structuré comme suit : le premier chapitre abordera le contexte général et l'analyse des besoins, le deuxième détaillera la phase de conception et de modélisation, tandis que le dernier chapitre mettra en avant la réalisation technique, l'interface utilisateur et les tests de l'application.

Chapitre 1

Présentation du projet

1.1 Description de l'application

CareDash est une application de bureau (Desktop) développée en **JavaFX** pour la gestion et le suivi des traitements médicaux. Elle est destinée aux professionnels de santé (médecins généralistes, spécialistes, cliniques) souhaitant numériser et centraliser la gestion de leurs patients.

L'application permet d'assurer :

- La gestion complète des dossiers patients (CRUD : Create, Read, Update, Delete).
- La prescription et le suivi des traitements médicamenteux.
- La visualisation en temps réel des statistiques du cabinet (nombre de patients, traitements actifs, ordonnances expirées).
- La génération d'ordonnances imprimables au format PDF.

L'architecture de l'application repose sur le patron de conception **MVC (Modèle-Vue-Contrôleur)**. La couche Modèle est constituée des classes **Patient** et **Treatment** qui représentent les entités métier. La couche Vue est définie à l'aide de fichiers **FXML** et d'une feuille de style **CSS** pour l'aspect graphique. La couche Contrôleur assure la liaison entre l'interface utilisateur et la logique métier, en utilisant **JDBC** pour interagir avec la base de données **MySQL**.

L'application se distingue par des fonctionnalités avancées telles que :

- La **coloration dynamique** des lignes de traitement expirées pour alerter le médecin.
- Un **système de blocage de sécurité** qui empêche la prescription d'un médicament en cas d'allergie détectée chez le patient.
- La **recherche en temps réel** via **FilteredList** qui offre une expérience utilisateur fluide et réactive.
- L'**impression d'ordonnances** directement depuis l'application.

CareDash répond ainsi aux besoins des professionnels de santé en leur offrant un outil complet, sécurisé et ergonomique pour la gestion quotidienne de leurs patients.

1.2 Besoins fonctionnels

Les besoins fonctionnels de l'application se déclinent en plusieurs modules distincts :

Authentification et gestion des comptes

- L'utilisateur doit pouvoir se connecter via un formulaire de connexion (email et mot de passe).
- L'utilisateur doit pouvoir créer un nouveau compte via un formulaire d'inscription (nom complet, email, mot de passe, confirmation).
- L'application doit afficher des messages d'erreur en cas d'identifiants incorrects ou de champs invalides.
- Le nom du médecin connecté doit être affiché dans l'interface principale.

Tableau de bord statistique

- L'application doit afficher le nombre total de patients enregistrés.
- L'application doit afficher le nombre de traitements actifs (date de fin non dépassée).
- L'application doit afficher le nombre d'ordonnances expirées (date de fin dépassée).
- L'application doit présenter un graphique circulaire (**PieChart**) illustrant la répartition des patients par sexe.
- L'application doit afficher la liste des 5 derniers patients enregistrés.

Gestion des patients (CRUD)

- L'utilisateur doit pouvoir ajouter un nouveau patient (nom, date de naissance, sexe, allergies).
- L'utilisateur doit pouvoir modifier les informations d'un patient existant.
- L'utilisateur doit pouvoir supprimer un patient (avec confirmation).
- L'utilisateur doit pouvoir rechercher un patient par nom via un champ de recherche dynamique.
- La date de naissance doit respecter le format YYYY-MM-DD.

Gestion des traitements / Ordonnances

- L'utilisateur doit pouvoir sélectionner un patient dans un menu déroulant (**ComboBox**).
- L'utilisateur doit pouvoir prescrire un nouveau traitement (médicament, type, posologie, date de fin).
- L'utilisateur doit pouvoir modifier ou supprimer un traitement existant.
- L'application doit colorer automatiquement en rouge les traitements expirés.
- L'application doit bloquer la prescription d'un médicament si le patient est allergique.

Génération d'ordonnance

- L'utilisateur doit pouvoir générer une ordonnance imprimable au format PDF.
- L'ordonnance doit contenir : l'en-tête institutionnel, les coordonnées du patient, la liste des médicaments prescrits, la posologie, la date de fin et la signature.

Navigation et déconnexion

- L'utilisateur doit pouvoir naviguer entre les différentes vues (Tableau de bord, Patients, Ordonnances).
- L'utilisateur doit pouvoir se déconnecter pour revenir à l'écran de connexion.
- L'interface doit afficher la date, le jour et l'heure en temps réel.

1.3 Besoins non fonctionnels

Performance et réactivité

- L'application doit afficher les données en temps réel sans latence perceptible.
- La recherche dynamique doit s'effectuer instantanément à chaque caractère saisi, sans rechargement de la base de données.
- L'utilisation de `FilteredList` garantit un filtrage en mémoire pour des performances optimales.

Sécurité

- Les requêtes SQL doivent utiliser des `PreparedStatement` pour prévenir les injections SQL.
- Le système doit vérifier automatiquement les allergies avant toute prescription.
- Les mots de passe doivent être masqués lors de la saisie.

Ergonomie et Interface utilisateur

- L'interface doit être épurée, intuitive et facile à prendre en main.
- Une charte graphique cohérente doit être appliquée via une feuille de style CSS.
- Les boutons doivent avoir des couleurs distinctes (vert pour Ajouter, orange pour Modifier, rouge pour Supprimer).
- L'application doit être responsive et s'adapter à différentes résolutions d'écran.

Maintenabilité et évolutivité

- L'architecture MVC assure une séparation claire des responsabilités.
- Les fichiers FXML permettent de modifier l'interface sans toucher au code Java.
- La base de données peut être étendue facilement avec de nouvelles tables ou colonnes.

Portabilité

- L'application est développée en Java, ce qui la rend compatible avec les systèmes d'exploitation Windows, macOS et Linux.
- La base de données MySQL est hébergée localement via XAMPP ou WampServer.

Disponibilité

- L'application fonctionne en mode local, garantissant une disponibilité permanente.

Chapitre 2

Conception

2.1 Architecture MVC

Pour assurer la robustesse, la maintenabilité et l'évolutivité de notre application **Ca-reDash**, nous avons adopté l'architecture **MVC (Modèle-Vue-Contrôleur)**. Ce patron de conception permet une séparation claire des responsabilités en divisant l'application en trois couches distinctes :

- **Le Modèle (Model)** : Il représente la couche de données et la logique métier. Dans notre projet, cette couche inclut les classes Java (**Patient.java**, **Treatment.java**) qui définissent la structure des entités ainsi que les classes DAO (*Data Access Object*) qui gèrent les interactions avec la base de données MySQL via JDBC. Il garantit l'intégrité des données et leur persistance.
- **La Vue (View)** : Elle correspond à l'interface utilisateur graphique. Développée en **FXML**, elle définit la structure visuelle de l'application (fenêtres, boutons, tableaux, champs de saisie). La Vue est purement déclarative et ne contient pas de logique de traitement, ce qui facilite la modification du design sans altérer le code métier.
- **Le Contrôleur (Controller)** : Il agit comme l'intermédiaire entre le Modèle et la Vue. Les classes contrôleurs (ex : **PatientController.java**, **TreatmentController.java**) interceptent les événements utilisateur (clics, saisies), sollicitent le modèle pour effectuer les calculs ou mises à jour nécessaires, et actualisent la vue en conséquence.

Cette architecture est particulièrement adaptée à **JavaFX**, car elle permet une liaison efficace (*data binding*) entre les composants graphiques et les données. Elle facilite ainsi le travail en équipe et permet d'isoler les bugs plus rapidement en restreignant les responsabilités de chaque module.

2.2 Diagramme de classes UML

Le diagramme de classes UML (Figure 2.2.1) présente la structure statique du système en détaillant les classes principales, leurs attributs, leurs méthodes et les relations qui les lient.

Ce diagramme met en évidence les éléments suivants :

- **La classe Patient** : elle encapsule les informations personnelles d'un patient (ID, nom, date de naissance, sexe, allergies). Elle sert de conteneur de données pour la vue et la base.

- **La classe `Treatment`** : elle représente une ordonnance ou un traitement médicamenteux. Elle est associée à un patient via une relation de composition (un patient peut avoir plusieurs traitements). Ses attributs incluent le médicament, le type, la posologie et la date de fin.
- **La classe `DatabaseManager`** : elle centralise la connexion JDBC et fournit des méthodes statiques pour exécuter des requêtes SQL (INSERT, SELECT, UPDATE, DELETE). Elle agit comme une façade d'accès aux données.
- **Les contrôleurs** : ils implémentent l'interface `Initializable` de JavaFX et contiennent des références vers les composants FXML (annotés `@FXML`). Ils utilisent les instances des classes modèles pour alimenter les tableaux et gérer les événements utilisateur.

Les flèches de dépendance montrent que les contrôleurs dépendent du `DatabaseManager` pour la persistance, tandis que les classes modèles sont indépendantes de la couche d'accès aux données.

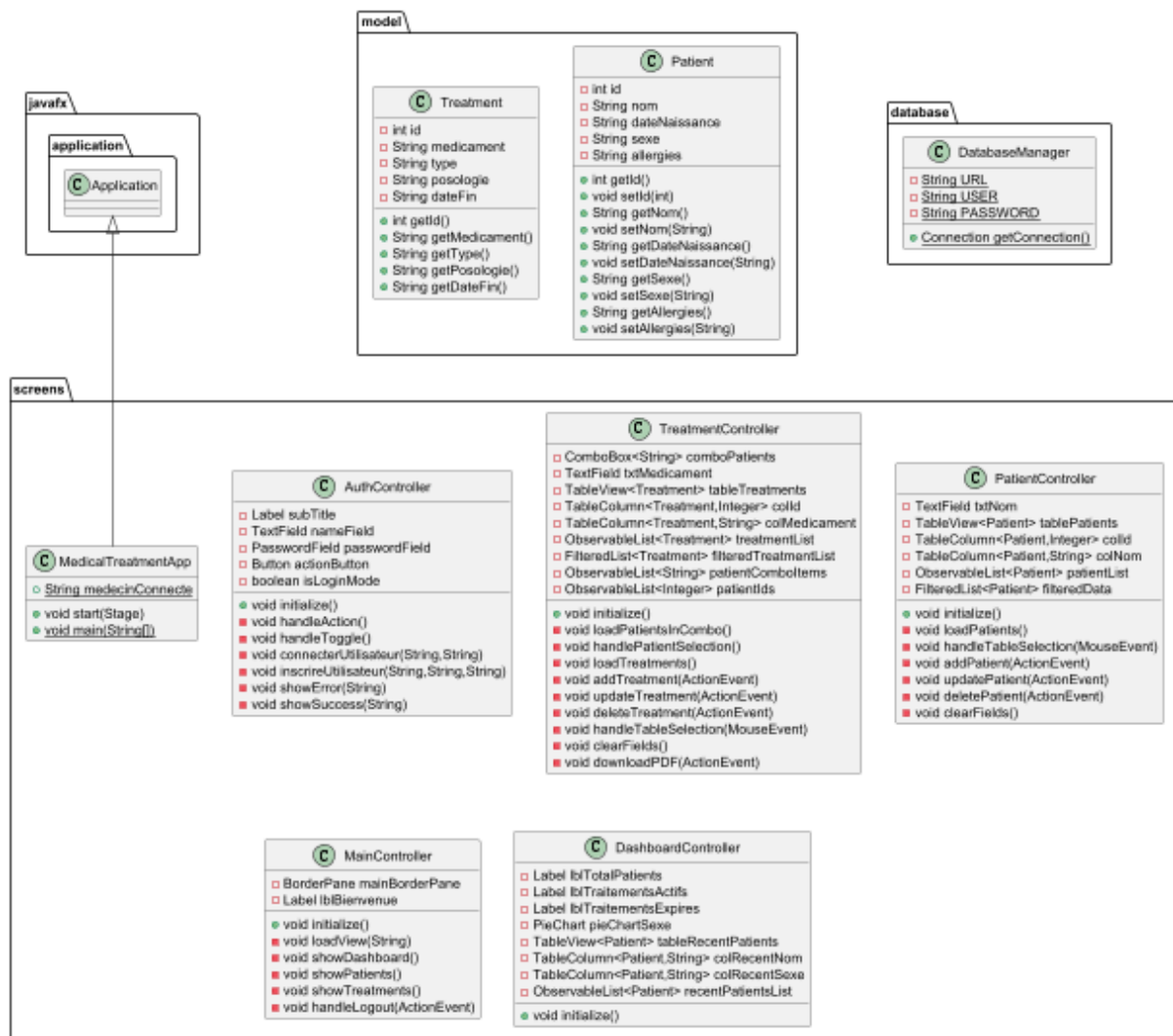


FIGURE 2.2.1 – Diagramme de classes

2.3 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation (Figure 2.3.1) représente les fonctionnalités offertes par l'application du point de vue de l'utilisateur, ainsi que les interactions entre les acteurs et le système.

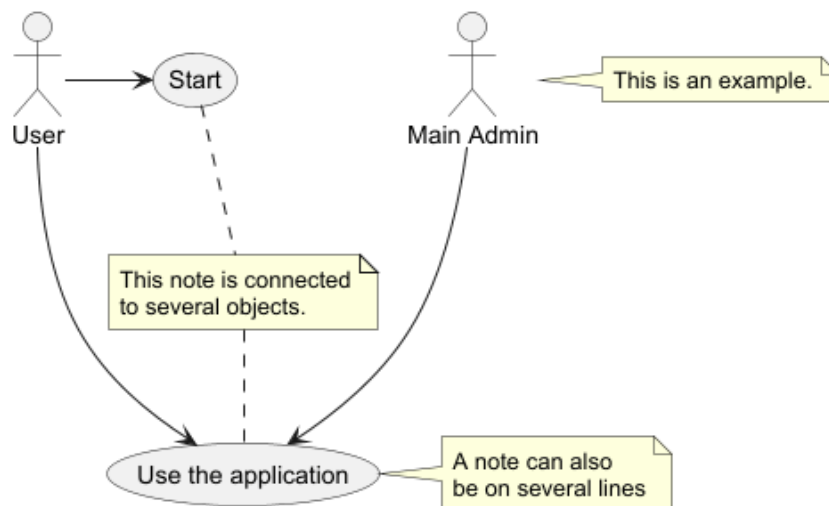


FIGURE 2.3.1 – Diagramme de cas d'utilisation de CareDash

Les principaux acteurs identifiés sont :

- **L'Utilisateur (User)** : il s'agit du médecin ou du professionnel de santé qui utilise l'application au quotidien. Il doit s'authentifier pour accéder aux fonctionnalités.
- **L'Administrateur (Admin)** : bien que non représenté dans la version actuelle, il pourrait gérer les comptes utilisateurs dans une version future. Dans notre périmètre, l'utilisateur unique est le médecin traitant.

Les cas d'utilisation majeurs sont les suivants :

- **S'authentifier** : l'utilisateur se connecte via l'écran de connexion ou crée un compte via l'écran d'inscription. C'est le point d'entrée obligatoire.
- **Consulter le tableau de bord** : l'utilisateur visualise les statistiques globales (nombre de patients, traitements actifs, répartition par sexe, liste des derniers patients enregistrés).
- **Gérer les patients** : l'utilisateur peut ajouter, modifier, supprimer ou rechercher des patients (CRUD complet) avec filtrage en temps réel.
- **Gérer les traitements / ordonnances** : l'utilisateur peut prescrire de nouveaux traitements, les associer à un patient spécifique, et visualiser l'historique des prescriptions.
- **Générer une ordonnance PDF** : l'utilisateur peut exporter une ordonnance au format PDF à partir des données saisies.

2.4 Diagramme de séquence

Le diagramme de séquence (Figure 2.4.1) illustre l'interaction temporelle entre les objets pour un scénario d'authentification typique. Ce diagramme montre comment les différentes couches (Vue, Contrôleur, Modèle) collaborent pour réaliser une fonctionnalité.

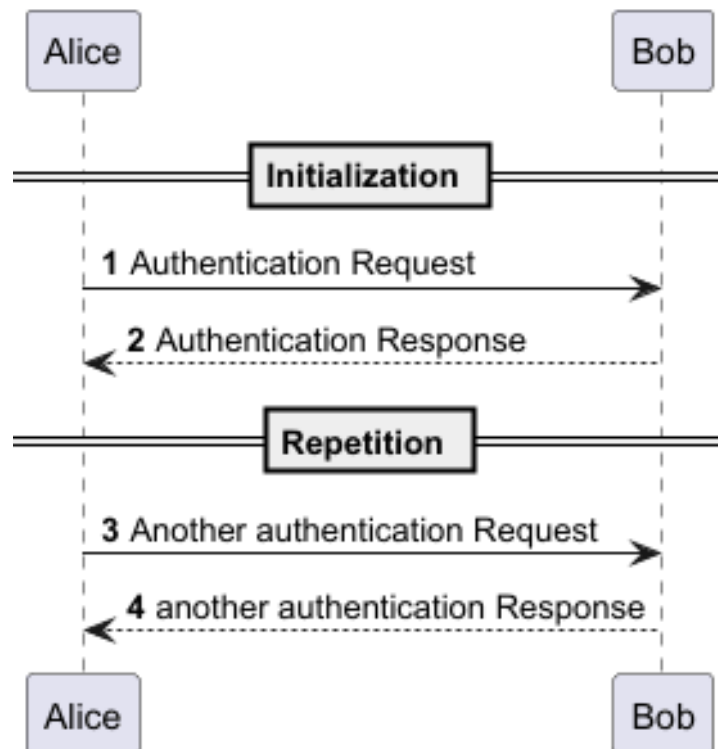


FIGURE 2.4.1 – Diagramme de séquence Authentification d'un utilisateur

Le scénario représenté se déroule comme suit :

1. L'utilisateur (acteur) saisit ses identifiants (email et mot de passe) dans l'interface `LoginView`.
2. Il clique sur le bouton « Se connecter », ce qui déclenche la méthode `handleLogin()` du contrôleur `AuthController`.
3. Le contrôleur appelle la méthode `authenticateUser(email, password)` de la classe `DatabaseManager` pour vérifier les informations dans la base de données MySQL.
4. `DatabaseManager` exécute une requête SQL `SELECT` et retourne un booléen (`true` si l'utilisateur existe, `false` sinon).
5. Si l'authentification est réussie, le contrôleur charge la vue principale `MainView.fxml` en remplaçant la scène actuelle, et transmet les données de l'utilisateur connecté.
6. Si l'authentification échoue, un message d'erreur est affiché dans la vue via une alerte JavaFX (`Alert.ERROR`).

Chapitre 3

Environnement de travail

3.1 Outils et technologies utilisés

Pour mener à bien le développement de l'application **CareDash**, nous avons utilisé un ensemble d'outils et de technologies adaptés au développement d'applications de bureau modernes :

- **Langage** : Java 8 / Java 11 (Adapté au support natif de JavaFX et à la compatibilité avec les bibliothèques JDBC).
- **IHM (Interface Homme-Machine)** : JavaFX avec moteur de rendu FXML pour la définition déclarative des interfaces, intégration de composants graphiques natifs (`TableView`, `Chart`, etc.) et feuilles de style CSS pour le design.
- **SGBD (Système de Gestion de Base de Données)** : MySQL (Moteur relationnel pour le stockage persistant des patients, traitements et ordonnances).
- **IDE (Environnement de Développement)** : JetBrains IntelliJ IDEA (édition Community), offrant une excellente intégration pour JavaFX et la gestion des dépendances.
- **Serveur local de base de données** : XAMPP / WampServer (Hébergeant le serveur MySQL s'exécutant sur le port standard 3306).
- **Gestion de version** : Git, avec un fichier `.gitignore` pour exclure les fichiers de compilation (`out/`, `.idea/`).

3.2 Structure du projet

Le projet **CareDash** est structuré selon une organisation modulaire respectant le patron de conception MVC (Modèle-Vue-Contrôleur). La capture ci-dessous (Figure 3.2.1) présente l'arborescence complète des packages et fichiers sources :

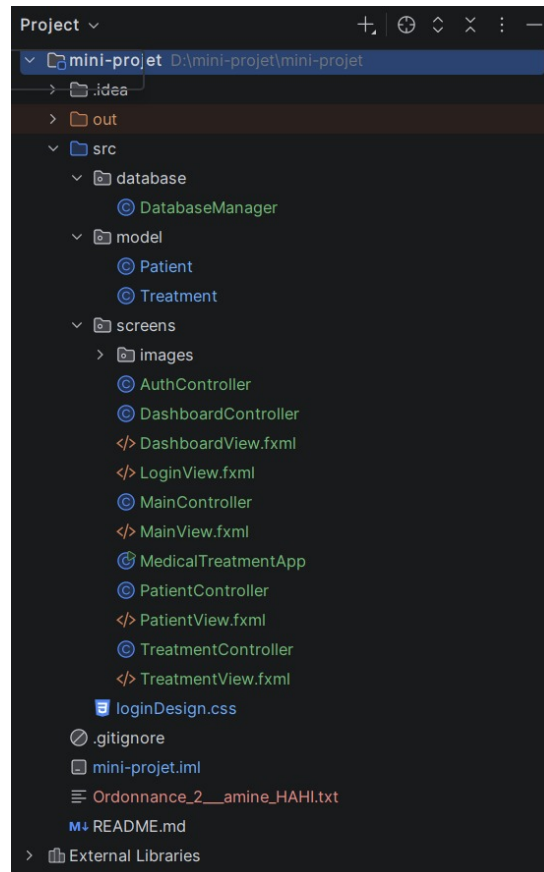


FIGURE 3.2.1 – Organisation arborescente des packages et fichiers du projet CareDash

L'architecture se décompose de manière précise en trois couches principales, visibles dans le dossier `src/` :

Couche Modèle et Accès aux Données (/src/database/)

Cette couche centralise toute la logique de persistance et la représentation des données métier :

- **DatabaseManager.java** : Classe utilitaire responsable de la gestion de la connexion JDBC à la base de données MySQL (chargement du driver, création de la session, fermeture des ressources).
- **Package model/** : Il contient les classes entités (Java Beans) qui reflètent la structure des tables de la base de données :
 - **Patient.java** : Définit les attributs d'un patient (ID, Nom, Date de naissance, Sexe, Allergies) avec ses getters/setters.
 - **Treatment.java** : Définit les attributs d'un traitement médical (ID, ID du patient associé, Médicament, Type, Posologie, Date de fin).

Couche Vue (Interfaces graphiques) /src/screens/

Ce répertoire regroupe l'ensemble des fichiers dédiés à l'affichage et au style de l'application :

- **Fichiers FXML (*View.fxml)** : Ce sont des fichiers XML décrivant la structure des interfaces utilisateur. On y retrouve :

- `LoginView.fxml` : Formulaire de connexion.
- `MainView.fxml` : Fenêtre principale avec le menu latéral et la zone de contenu.
- `DashboardView.fxml` : Vue du tableau de bord statistique (KPIs et graphiques).
- `PatientView.fxml` : Vue de gestion des patients (tableau, champs de recherche et filtres).
- `TreatmentView.fxml` : Vue de gestion des ordonnances et traitements.
- **Feuille de style CSS (`loginDesign.css`)** : Centralise les règles de mise en forme (couleurs, polices, marges) pour uniformiser la charte graphique médicale de l'application.
- **Dossier `images/`** : Contient les ressources iconographiques (logos, icônes des boutons) utilisées dans l'interface.

Couche Contrôleur et Point d'entrée `/src/screens/`

Cette couche assure la liaison entre les événements utilisateur et les données :

- **Contrôleurs JavaFX (classes en `*Controller.java`)** : Ils interceptent les interactions de l'utilisateur (clics, saisies) et mettent à jour les vues. La liste exhaustive des contrôleurs est la suivante :
 - `AuthController.java` : Gère l'authentification (connexion et inscription).
 - `MainController.java` : Gère le chargement des différentes vues dans la zone centrale de la fenêtre principale (navigation entre Dashboard, Patients et Ordonnances).
 - `DashboardController.java` : Alimente les indicateurs et les graphiques du tableau de bord avec les données issues de la base.
 - `PatientController.java` : Gère les opérations CRUD (Ajouter, Modifier, Supprimer) et la recherche dynamique sur les patients.
 - `TreatmentController.java` : Gère le filtrage par patient, la saisie des ordonnances et la génération du PDF.
- **Classe principale (`MedicalTreatmentApp.java`)** : C'est la classe de lancement de l'application JavaFX. Elle étend `Application` et surcharge la méthode `start()` pour charger le premier écran (Login) et initialiser la scène principale.

Éléments complémentaires à la racine du projet

En dehors du dossier `src/`, on trouve également :

- `README.md` : Fichier de documentation décrivant l'installation et les fonctionnalités du projet.
- `mini-project.iml` : Fichier de configuration du module IntelliJ.
- `Ordonnance_2__amine_HAHI.txt` : Exemple de fichier texte généré lors de l'export d'une ordonnance (preuve du fonctionnement de la fonctionnalité d'export).
- Dossiers de compilation (`out/`) et de configuration IDE (`.idea/`) qui sont généralement ignorés par Git.

Cette organisation rigoureuse permet une séparation claire des responsabilités, facilitant la maintenance, les tests unitaires et l'ajout de nouvelles fonctionnalités sans impacter l'existant.

Chapitre 4

Répartition des tâches

Afin de mener à bien ce projet dans les délais impartis, nous avons opté pour une répartition des tâches basée sur les compétences de chacun.

Chaimae El maachi s'est principalement consacrée à la partie *Design et Interface Utilisateur (IHM)* : conception des vues FXML, mise en place de la charte graphique CSS, intégration des ressources visuelles. Elle a également participé à la conception de la base de données ainsi qu'au développement des opérations CRUD pour la gestion des patients.

Boughida Najoua a pris en charge la couche *Contrôleur et Logique Métier* : gestion de la base de données (JDBC), création des classes modèles, développement des contrôleurs JavaFX (authentification, CRUD, traitement, export PDF).

Note importante : L'intégralité du document final a été relue, enrichie et corrigée ensemble. Ce travail reflète donc une véritable collaboration entre les deux coéquipières, chaque chapitre ayant bénéficié des apports et des relectures de l'autre.

4.1 Tâches de l'étudiant 1 – Chaimae El maachi

Tâche réalisée	Fichier(s) concerné(s)	Statut
Conception des interfaces graphiques (FXML) : connexion, inscription, tableau de bord, patients, traitements	screens/LoginView.fxml, screens/MainView.fxml, screens/DashboardView.fxml, screens/PatientView.fxml, screens/TreatmentView.fxml	Terminé
Mise en place de la charte graphique et du style CSS	screens/loginDesign.css	Terminé
Intégration des ressources graphiques (logos, icônes)	screens/images/ (tous les fichiers)	Terminé
Conception de la base de données et participation au développement CRUD des patients	database/DatabaseManager.java, database/model/Patient.java et script SQL	Terminé
Design de la mise en page du tableau de bord (disposition des KPIs, graphiques et cartes)	screens/DashboardView.fxml (partie design)	Terminé
Rédaction du README.md (documentation technique du projet)	README.md	Terminé

TABLE 4.1 – Tâches réalisées par l'étudiant 1 – Chaimae El maachi

4.2 Tâches de l'étudiant 2 – Boughida Najoua

Tâche réalisée	Fichier(s) concerné(s)	Statut
Mise en place de la base de données et du gestionnaire de connexion JDBC	database/DatabaseManager.java et script SQL	Terminé
Création des classes modèles (entités Patient et Treatment)	database/model/Patient.java, database/model/Treatment.java	Terminé
Implémentation des contrôleurs de navigation et d'authentification	screens/AuthController.java, screens/MainController.java	Terminé
Développement du contrôleur Patient (CRUD, recherche en temps réel)	screens/PatientController.java	Terminé
Développement du contrôleur Traitement (prescription, filtrage, génération PDF/TXT)	screens/TreatmentController.java, Ordonnance_*.txt	Terminé
Participation à la rédaction et à la relecture du rapport (Remerciements, Introduction, Conclusion)	Rapport_Projet_CareDash.pdf	Terminé

TABLE 4.2 – Tâches réalisées par l'étudiant 2 – Boughida Najoua

Chapitre 5

Explication du code

Ce chapitre présente une analyse détaillée des différentes couches de l'application **CarreDash**. Nous décrivons successivement la gestion de la base de données, les classes modèles et les contrôleurs JavaFX qui implémentent la logique métier.

5.1 Connexion à la base de données – `DatabaseManager.java`

La classe `DatabaseManager.java`, située dans le package `database/`, est responsable de l'établissement de la connexion à la base de données MySQL.

Listing 5.1 – `DatabaseManager.java`

```
1 package database;
2
3 import java.sql.Connection;
4 import java.sql.DriverManager;
5 import java.sql.SQLException;
6
7 public class DatabaseManager {
8     private static final String URL = "jdbc:mysql://localhost
9         :3306/medical_db";
10    private static final String USER = "root";
11    private static final String PASSWORD = "";
12
13    public static Connection getConnection() throws SQLException
14    {
15        return DriverManager.getConnection(URL, USER, PASSWORD);
16    }
17 }
```

Cette classe utilise une approche simple et efficace :

- `URL` : Chaîne de connexion JDBC pointant vers la base `medical_db` sur `localhost:3306`.
- `USER` : Nom d'utilisateur MySQL (`root` par défaut sous XAMPP).
- `PASSWORD` : Mot de passe MySQL (vide par défaut).
- `getConnection()` : Établit et retourne une nouvelle connexion à chaque appel.

Cette conception permet une gestion légère et évite les problèmes de concurrence liés aux connexions partagées.

5.2 Les classes modèles

Les classes modèles sont des *JavaBeans* situées dans le package `model/`. Elles représentent les entités métier de l'application.

Classe `Patient.java`

Cette classe modélise un patient avec les attributs suivants :

- `id` : Identifiant unique (auto-incrémenté en base).
- `nom` : Nom complet du patient.
- `dateNaissance` : Date de naissance (stockée sous forme de chaîne `String`).
- `sexe` : Caractère 'M' ou 'F'.
- `allergies` : Texte décrivant les allergies connues.

Un constructeur paramétré permet de créer facilement des instances à partir des données de la base.

Classe `Treatment.java`

Cette classe modélise un traitement médical avec les attributs suivants :

- `id` : Identifiant unique.
- `patientId` : Clé étrangère référençant le patient associé.
- `medicament` : Nom du médicament prescrit.
- `type` : Catégorie ou type de médicament.
- `posologie` : Dosage et fréquence.
- `dateFin` : Date de fin du traitement.

5.3 Les contrôleurs

Les contrôleurs JavaFX assurent la liaison entre l'interface utilisateur (FXML) et la logique métier. Ils sont situés dans le package `screens/`.

`MedicalTreatmentApp.java` – Point d'entrée

Cette classe est le point d'entrée de l'application JavaFX. Elle étend `Application` et surcharge la méthode `start()` :

Listing 5.2 – `MedicalTreatmentApp.java`

```
1 public class MedicalTreatmentApp extends Application {
2     public static String medecinConnecte = "Docteur";
3
4     @Override
5     public void start(Stage primaryStage) throws Exception {
6         Parent root = FXMLLoader.load(getClass().getResource("
7             LoginView.fxml"));
8         primaryStage.setTitle("Suivi_Médical_─Authentification")
9             ;
10        primaryStage.setScene(new Scene(root, 380, 420));
11        primaryStage.setResizable(false);
12        primaryStage.show();
13    }
```

```

11     }
12 }

```

La variable statique `medecinConnecte` stocke le nom du médecin connecté pour personnaliser l'interface.

AuthController.java – Authentification

Ce contrôleur gère les écrans de connexion et d'inscription. Il permet :

- La vérification des identifiants saisis dans `LoginView.fxml`.
- La validation des champs du formulaire d'inscription.
- La navigation entre les deux écrans.
- Le stockage du nom du médecin connecté dans `MedicalTreatmentApp.medecinConnecte`.

MainController.java – Navigation principale

Ce contrôleur gère la fenêtre principale de l'application (`MainView.fxml`) :

- **Initialisation** : La méthode `initialize()` personnalise le message de bienvenue avec le nom du médecin connecté et démarre un chronomètre (`Timeline`) pour afficher la date et l'heure en temps réel.
- **Navigation** : Les méthodes `showDashboard()`, `showPatients()` et `showTreatments()` chargent les vues correspondantes dans la zone centrale (`BorderPane`).
- **Déconnexion** : `handleLogout()` charge à nouveau l'écran de connexion.

Listing 5.3 – Chargement dynamique des vues

```

1 private void loadView(String fxml) {
2     try {
3         Parent root = FXMLLoader.load(getClass().getResource(fxml));
4         mainBorderPane.setCenter(root);
5     } catch (Exception e) {
6         e.printStackTrace();
7     }
8 }

```

DashboardController.java – Tableau de bord

Ce contrôleur alimente le tableau de bord avec des données statistiques :

- **Indicateurs KPIs** :
 - Nombre total de patients (`SELECT COUNT(*) FROM patient`).
 - Traitements actifs (`date_fin >= today`).
 - Traitements expirés (`date_fin < today`).
- **Graphique circulaire** : Répartition des patients par sexe (Hommes / Femmes) avec un `PieChart`.
- **Patients récents** : Affichage des 5 derniers patients enregistrés dans un `TableView`.

PatientController.java – Gestion des patients

Ce contrôleur implémente les opérations CRUD complètes pour les patients :

Initialisation de la vue

Listing 5.4 – Initialisation du PatientController

```
1 public void initialize() {
2     // Liaison des colonnes du TableView
3     colId.setCellValueFactory(new PropertyValueFactory<>("id"));
4     colNom.setCellValueFactory(new PropertyValueFactory<>("nom"));
5     ;
6     colDate.setCellValueFactory(new PropertyValueFactory<>("
    dateNaissance"));
7     colSexe.setCellValueFactory(new PropertyValueFactory<>("sexe"
    ));
8     colAllergies.setCellValueFactory(new PropertyValueFactory<>("
    allergies"));
9
10    loadPatients(); // Chargement initial
11
12    // Filtrage dynamique
13    filteredData = new FilteredList<>(patientList, p -> true);
14    txtRecherche.textProperty().addListener((obs, old, newVal) ->
15    {
16        filteredData.setPredicate(patient -> {
17            if (newVal == null || newVal.isEmpty()) return true;
18            return patient.getNom().toLowerCase().contains(newVal
19            .toLowerCase().trim());
20        });
21    });
22    tablePatients.setItems(filteredData);
23 }
```

Chargement des patients

La méthode `loadPatients()` interroge la base de données et remplit la `ObservableList` :

Listing 5.5 – Chargement des patients

```
1 private void loadPatients() {
2     patientList.clear();
3     String sql = "SELECT * FROM patient";
4     try (Connection conn = DatabaseManager.getConnection();
5         Statement stmt = conn.createStatement();
6         ResultSet rs = stmt.executeQuery(sql)) {
7         while (rs.next()) {
8             patientList.add(new Patient(
9                 rs.getInt("id"),
10                rs.getString("nom"),
11                rs.getString("date_naissance"),
12                rs.getString("sexe"),
13                rs.getString("allergies")
14            ));
15        }
16    }
```

```

16     } catch (Exception e) { e.printStackTrace(); }
17 }

```

Opérations CRUD

- **Ajouter (addPatient)** : Valide les champs (nom non vide, date au format YYYY-MM-DD) et exécute une requête INSERT.
- **Modifier (updatePatient)** : Met à jour le patient sélectionné avec une requête UPDATE.
- **Supprimer (deletePatient)** : Supprime le patient sélectionné avec une requête DELETE.

Toutes les opérations utilisent des `PreparedStatement` pour prévenir les injections SQL.

TreatmentController.java – Gestion des traitements

Ce contrôleur gère l'interface des ordonnances avec des fonctionnalités avancées :

Chargement des patients dans le ComboBox

Listing 5.6 – Chargement des patients dans le ComboBox

```

1 private void loadPatientsInCombo() {
2     patientComboItems.clear(); patientIds.clear();
3     try (Connection conn = DatabaseManager.getConnection();
4         Statement stmt = conn.createStatement();
5         ResultSet rs = stmt.executeQuery("SELECT id, nom FROM
6         patient")) {
7         while (rs.next()) {
8             patientIds.add(rs.getInt("id"));
9             patientComboItems.add(rs.getInt("id") + "-" + rs.
10                getString("nom"));
11         }
12         comboPatients.setItems(patientComboItems);
13     } catch (Exception e) { e.printStackTrace(); }
14 }

```

Coloration dynamique des lignes expirées

Les traitements dont la date de fin est dépassée apparaissent en rouge :

Listing 5.7 – Coloration des lignes expirées

```

1 tableTreatments.setRowFactory(tv -> new TableRow<Treatment>() {
2     @Override
3     protected void updateItem(Treatment item, boolean empty) {
4         super.updateItem(item, empty);
5         if (item != null && !empty) {
6             LocalDate dateFin = LocalDate.parse(item.getDateFin())
7                 );
8             if (dateFin.isBefore(LocalDate.now())) {

```

```

8         setStyle("-fx-background-color:_#ffcccc;_fx-text
          -fill:_#c0392b;_fx-font-weight:_bold;");
9     } else { setStyle(""); }
10 }
11 }
12 });

```

Blocage de sécurité – Vérification des allergies

Avant d'ajouter un traitement, le système vérifie si le patient est allergique au médicament :

Listing 5.8 – Vérification des allergies

```

1 String alg = rs.getString("allergies");
2 if (alg != null && !alg.trim().isEmpty()) {
3     for (String a : alg.split(",")) {
4         if (a.trim().equalsIgnoreCase(med)) {
5             Alert alert = new Alert(Alert.AlertType.ERROR);
6             alert.setTitle("Blocage de sécurité");
7             alert.setHeaderText("Allergie détectée!");
8             alert.setContentText("Le patient est allergique à : "
9                                 + a.trim());
10            alert.showAndWait();
11            return;
12        }
13    }

```

Génération d'ordonnance (impression)

- La méthode `downloadPDF()` construit dynamiquement une ordonnance imprimable :
- Création d'un conteneur `VBox` avec mise en page A4.
 - En-tête institutionnel (Centre Médical International, Dr. nom).
 - Liste des médicaments prescrits dans une `GridPane`.
 - Impression via `PrinterJob`.

5.4 Fonctionnalités spécifiques

Filtrage dynamique avec `FilteredList`

La recherche en temps réel est implémentée dans `PatientController` et `TreatmentController` :

Listing 5.9 – Filtrage dynamique

```

1 filteredData = new FilteredList<>(patientList, p -> true);
2 txtRecherche.textProperty().addListener((obs, old, newVal) -> {
3     filteredData.setPredicate(patient -> {
4         if (newVal == null || newVal.isEmpty()) return true;
5         return patient.getNom().toLowerCase().contains(newVal.
            toLowerCase().trim());

```

```

6     });
7 });
8 tablePatients.setItems(filteredData);

```

Chronomètre en temps réel

MainController utilise une Timeline pour afficher la date et l'heure dynamiquement :

Listing 5.10 – Affichage de l'heure en temps réel

```

1 Timeline timeline = new Timeline(
2     new KeyFrame(Duration.seconds(1), e -> {
3         LocalDateTime now = LocalDateTime.now();
4         dateLabel.setText(now.format(DateTimeFormatter.ofPattern(
5             "dd/MM/yyyy"))));
6         dayLabel.setText(now.format(DateTimeFormatter.ofPattern("
7             EEEE"))));
8         timeLabel.setText(now.format(DateTimeFormatter.ofPattern(
9             "HH:mm:ss"))));
10    })
11 );
12 timeline.setCycleCount(Timeline.INDEFINITE);
13 timeline.play();

```

Génération d'ordonnance imprimable

L'application intègre un système d'impression d'ordonnances via `PrinterJob`, permettant de générer un document structuré avec :

- En-tête institutionnel avec le nom du médecin.
- Coordonnées du patient.
- Tableau des médicaments prescrits.
- Signature et cachet.

Chapitre 6

Explication du code

Ce chapitre présente une analyse détaillée des différentes couches de l'application **CarreDash**. Nous décrivons successivement la gestion de la base de données, les classes modèles, les contrôleurs JavaFX et enfin les fichiers FXML qui définissent les interfaces utilisateur.

6.1 Connexion à la base de données – `DatabaseManager.java`

La classe `DatabaseManager.java`, située dans le package `database/`, est responsable de l'établissement de la connexion à la base de données MySQL.

Listing 6.1 – `DatabaseManager.java`

```
1 package database;
2
3 import java.sql.Connection;
4 import java.sql.DriverManager;
5 import java.sql.SQLException;
6
7 public class DatabaseManager {
8     private static final String URL = "jdbc:mysql://localhost
9         :3306/medical_db";
10    private static final String USER = "root";
11    private static final String PASSWORD = "";
12
13    public static Connection getConnection() throws SQLException
14    {
15        return DriverManager.getConnection(URL, USER, PASSWORD);
16    }
17 }
```

Cette classe utilise une approche simple et efficace :

- `URL` : Chaîne de connexion JDBC pointant vers la base `medical_db` sur `localhost:3306`.
- `USER` : Nom d'utilisateur MySQL (`root` par défaut sous XAMPP).
- `PASSWORD` : Mot de passe MySQL (vide par défaut).
- `getConnection()` : Établit et retourne une nouvelle connexion à chaque appel.

Cette conception permet une gestion légère et évite les problèmes de concurrence liés aux connexions partagées.

6.2 Les classes modèles

Les classes modèles sont des *JavaBeans* situées dans le package `model/`. Elles représentent les entités métier de l'application.

Classe `Patient.java`

Cette classe modélise un patient avec les attributs suivants :

- `id` : Identifiant unique (auto-incrémenté en base).
- `nom` : Nom complet du patient.
- `dateNaissance` : Date de naissance (stockée sous forme de chaîne `String`).
- `sexe` : Caractère 'M' ou 'F'.
- `allergies` : Texte décrivant les allergies connues.

Un constructeur paramétré permet de créer facilement des instances à partir des données de la base.

Classe `Treatment.java`

Cette classe modélise un traitement médical avec les attributs suivants :

- `id` : Identifiant unique.
- `patientId` : Clé étrangère référençant le patient associé.
- `medicament` : Nom du médicament prescrit.
- `type` : Catégorie ou type de médicament.
- `posologie` : Dosage et fréquence.
- `dateFin` : Date de fin du traitement.

6.3 Les contrôleurs

Les contrôleurs JavaFX assurent la liaison entre l'interface utilisateur (FXML) et la logique métier. Ils sont situés dans le package `screens/`.

`MedicalTreatmentApp.java` – Point d'entrée

Cette classe est le point d'entrée de l'application JavaFX. Elle étend `Application` et surcharge la méthode `start()` :

Listing 6.2 – `MedicalTreatmentApp.java`

```
1 public class MedicalTreatmentApp extends Application {
2     public static String medecinConnecte = "Docteur";
3
4     @Override
5     public void start(Stage primaryStage) throws Exception {
6         Parent root = FXMLLoader.load(getClass().getResource("
7             LoginView.fxml"));
8         primaryStage.setTitle("Suivi_Médical_─Authentification")
9             ;
10        primaryStage.setScene(new Scene(root, 380, 420));
11        primaryStage.setResizable(false);
12        primaryStage.show();
13    }
```

```

11     }
12 }

```

La variable statique `medecinConnecte` stocke le nom du médecin connecté pour personnaliser l'interface.

AuthController.java – Authentification

Ce contrôleur gère les écrans de connexion et d'inscription. Il permet :

- La vérification des identifiants saisis dans `LoginView.fxml`.
- La validation des champs du formulaire d'inscription.
- La navigation entre les deux écrans.
- Le stockage du nom du médecin connecté dans `MedicalTreatmentApp.medecinConnecte`.

MainController.java – Navigation principale

Ce contrôleur gère la fenêtre principale de l'application (`MainView.fxml`) :

- **Initialisation** : La méthode `initialize()` personnalise le message de bienvenue avec le nom du médecin connecté et démarre un chronomètre (`Timeline`) pour afficher la date et l'heure en temps réel.
- **Navigation** : Les méthodes `showDashboard()`, `showPatients()` et `showTreatments()` chargent les vues correspondantes dans la zone centrale (`BorderPane`).
- **Déconnexion** : `handleLogout()` charge à nouveau l'écran de connexion.

Listing 6.3 – Chargement dynamique des vues

```

1 private void loadView(String fxml) {
2     try {
3         Parent root = FXMLLoader.load(getClass().getResource(fxml));
4         mainBorderPane.setCenter(root);
5     } catch (Exception e) {
6         e.printStackTrace();
7     }
8 }

```

DashboardController.java – Tableau de bord

Ce contrôleur alimente le tableau de bord avec des données statistiques :

- **Indicateurs KPIs** :
 - Nombre total de patients (`SELECT COUNT(*) FROM patient`).
 - Traitements actifs (`date_fin >= today`).
 - Traitements expirés (`date_fin < today`).
- **Graphique circulaire** : Répartition des patients par sexe (Hommes / Femmes) avec un `PieChart`.
- **Patients récents** : Affichage des 5 derniers patients enregistrés dans un `TableView`.

PatientController.java – Gestion des patients

Ce contrôleur implémente les opérations CRUD complètes pour les patients :

Initialisation de la vue

Listing 6.4 – Initialisation du PatientController

```
1 public void initialize() {
2     // Liaison des colonnes du TableView
3     colId.setCellValueFactory(new PropertyValueFactory<>("id"));
4     colNom.setCellValueFactory(new PropertyValueFactory<>("nom"));
5     ;
6     colDate.setCellValueFactory(new PropertyValueFactory<>("
    dateNaissance"));
7     colSexe.setCellValueFactory(new PropertyValueFactory<>("sexe"
    ));
8     colAllergies.setCellValueFactory(new PropertyValueFactory<>("
    allergies"));
9
10    loadPatients(); // Chargement initial
11
12    // Filtrage dynamique
13    filteredData = new FilteredList<>(patientList, p -> true);
14    txtRecherche.textProperty().addListener((obs, old, newVal) ->
15    {
16        filteredData.setPredicate(patient -> {
17            if (newVal == null || newVal.isEmpty()) return true;
18            return patient.getNom().toLowerCase().contains(newVal
19                .toLowerCase().trim());
20        });
21    });
22    tablePatients.setItems(filteredData);
23 }
```

Chargement des patients

La méthode `loadPatients()` interroge la base de données et remplit la `ObservableList` :

Listing 6.5 – Chargement des patients

```
1 private void loadPatients() {
2     patientList.clear();
3     String sql = "SELECT * FROM patient";
4     try (Connection conn = DatabaseManager.getConnection();
5         Statement stmt = conn.createStatement();
6         ResultSet rs = stmt.executeQuery(sql)) {
7         while (rs.next()) {
8             patientList.add(new Patient(
9                 rs.getInt("id"),
10                rs.getString("nom"),
11                rs.getString("date_naissance"),
12                rs.getString("sexe"),
13                rs.getString("allergies")
14            ));
15        }
16    }
```

```

16     } catch (Exception e) { e.printStackTrace(); }
17 }

```

Opérations CRUD

- **Ajouter (addPatient)** : Valide les champs (nom non vide, date au format YYYY-MM-DD) et exécute une requête INSERT.
- **Modifier (updatePatient)** : Met à jour le patient sélectionné avec une requête UPDATE.
- **Supprimer (deletePatient)** : Supprime le patient sélectionné avec une requête DELETE.

Toutes les opérations utilisent des `PreparedStatement` pour prévenir les injections SQL.

TreatmentController.java – Gestion des traitements

Ce contrôleur gère l'interface des ordonnances avec des fonctionnalités avancées :

Chargement des patients dans le ComboBox

Listing 6.6 – Chargement des patients dans le ComboBox

```

1 private void loadPatientsInCombo() {
2     patientComboItems.clear(); patientIds.clear();
3     try (Connection conn = DatabaseManager.getConnection();
4         Statement stmt = conn.createStatement();
5         ResultSet rs = stmt.executeQuery("SELECT _id, _nom FROM _
6         patient")) {
7         while (rs.next()) {
8             patientIds.add(rs.getInt("id"));
9             patientComboItems.add(rs.getInt("id") + "_-" + rs.
10                getString("nom"));
11        }
12        comboPatients.setItems(patientComboItems);
13    } catch (Exception e) { e.printStackTrace(); }
14 }

```

Coloration dynamique des lignes expirées

Les traitements dont la date de fin est dépassée apparaissent en rouge :

Listing 6.7 – Coloration des lignes expirées

```

1 tableTreatments.setRowFactory(tv -> new TableRow<Treatment>() {
2     @Override
3     protected void updateItem(Treatment item, boolean empty) {
4         super.updateItem(item, empty);
5         if (item != null && !empty) {
6             LocalDate dateFin = LocalDate.parse(item.getDateFin()
7                 );
8             if (dateFin.isBefore(LocalDate.now())) {

```

```

8         setStyle("-fx-background-color: #ffcccc; -fx-text
          -fill: #c0392b; -fx-font-weight: bold;");
9     } else { setStyle(""); }
10 }
11 }
12 });

```

Blocage de sécurité – Vérification des allergies

Avant d'ajouter un traitement, le système vérifie si le patient est allergique au médicament :

Listing 6.8 – Vérification des allergies

```

1 String alg = rs.getString("allergies");
2 if (alg != null && !alg.trim().isEmpty()) {
3     for (String a : alg.split(",")) {
4         if (a.trim().equalsIgnoreCase(med)) {
5             Alert alert = new Alert(Alert.AlertType.ERROR);
6             alert.setTitle("Blocage de sécurité");
7             alert.setHeaderText("Allergie détectée!");
8             alert.setContentText("Le patient est allergique à : "
9                 + a.trim());
10            alert.showAndWait();
11            return;
12        }
13    }

```

Génération d'ordonnance (impression)

- La méthode `downloadPDF()` construit dynamiquement une ordonnance imprimable :
- Création d'un conteneur `VBox` avec mise en page A4.
 - En-tête institutionnel (Centre Médical International, Dr. nom).
 - Liste des médicaments prescrits dans une `GridPane`.
 - Impression via `PrinterJob`.

6.4 Les fichiers FXML

Les fichiers FXML sont des documents XML qui décrivent la structure des interfaces utilisateur. Ils permettent une séparation claire entre la logique (contrôleurs) et la présentation (vue). Chaque vue est associée à un contrôleur JavaFX via l'attribut `fx:controller`.

LoginView.fxml – Écran d'authentification

- Ce fichier définit l'interface de connexion et d'inscription. Sa structure est la suivante :
- Un conteneur principal `VBox` avec un espacement de 15px et des marges intérieures de 30px.

- Un `ImageView` affichant le logo de l'application (`q1-removebg-preview.png`).
 - Un `Label` « Espace Connexion » en bleu (`#2c7dd4`) avec la police Berlin Sans FB.
 - Quatre champs de saisie :
 - `nameField` : invisible par défaut (pour l'inscription).
 - `emailField` : champ email.
 - `passwordField` : champ mot de passe (masqué).
 - `confirmPasswordField` : invisible par défaut (pour la confirmation).
 - Un bouton d'action (`actionButton`) avec le texte « Se connecter » et un style bleu (`#3498db`).
 - Un bouton de bascule (`toggleButton`) pour passer entre Connexion et Inscription.
 - Un `Label` de statut pour afficher les messages d'erreur ou de succès.
- La feuille de style `loginDesign.css` est appliquée via `stylesheets="@../loginDesign.css"`.

MainView.fxml – Fenêtre principale

Ce fichier structure la fenêtre principale avec un `BorderPane` :

- **Panneau gauche** : Un `VBox` avec une largeur de 220px contenant :
 - Le titre « Care Dash » avec la police Berlin Sans FB Demi Bold.
 - Trois boutons de navigation : « Tableau de Bord », « Patients » et « Ordonnances » (liés aux méthodes `showDashboard()`, `showPatients()`, `showTreatments()`).
 - Un bouton « Déconnexion » en bas (lié à `handleLogout()`).
- **Zone centrale** : Un `VBox` centré avec :
 - Une ligne `HBox` affichant la date, le jour et l'heure en temps réel (liés aux `Label` `dateLabel`, `dayLabel`, `timeLabel`).
 - Un `ImageView` affichant le logo `logo2-removebg-preview.png`.
 - Un message de bienvenue personnalisé (`lblBienvenue`) avec le nom du médecin connecté.
 - Des messages d'accueil génériques.

DashboardView.fxml – Tableau de bord

Ce fichier définit l'interface du tableau de bord statistique :

- Un `VBox` principal avec un fond `#ecf0f1` et un espacement de 30px.
- Un titre « Tableau de Bord Activité » en gras.
- Une ligne `HBox` contenant trois `VBox` (cartes) pour les KPIs :
 - `lblTotalPatients` : Total des patients.
 - `lblTraitementsActifs` : Traitements actifs.
 - `lblTraitementsExpires` : Traitements expirés.
- Une ligne `HBox` avec deux colonnes :
 - Un `PieChart` (`pieChartSexe`) pour la répartition des patients par sexe.
 - Un `TableView` (`tableRecentPatients`) avec deux colonnes : `colRecentNom` et `colRecentSexe`.

PatientView.fxml – Gestion des patients

Ce fichier organise l'interface CRUD des patients :

- Un `VBox` avec un fond `#ecf0f1` et un espacement de 15px.

- Un titre « Gestion des Patients ».
- Une barre de recherche avec `txtRecherche`.
- Une ligne `HBox` avec les champs de saisie :
 - `txtNom` : Nom complet.
 - `txtDate` : Date de naissance (AAAA-MM-JJ).
 - `txtSexe` : Sexe (M/F).
 - `txtAllergies` : Allergies.
- Trois boutons d'action : « Ajouter » (`addPatient`), « Modifier » (`updatePatient`), « Supprimer » (`deletePatient`).
- Un `TableView` (`tablePatients`) avec cinq colonnes : ID, Nom, Naissance, Sexe, Allergies.

TreatmentView.fxml – Gestion des ordonnances

Ce fichier structure l'interface des prescriptions :

- Un `VBox` avec un fond `#ecf0f1` et un espacement de 15px.
- Un titre « Gestion des Ordonnances ».
- Une barre de sélection du patient avec `comboPatients` (lié à `handlePatientSelection()`).
- Une barre de recherche contextuelle (`txtSearch`).
- Une ligne `HBox` avec les champs de saisie des traitements :
 - `txtMedicament` : Nom du médicament.
 - `txtType` : Type.
 - `txtPosologie` : Posologie.
 - `txtDateFin` : Date de fin (AAAA-MM-JJ).
- Trois boutons d'action : « Prescrire » (`addTreatment`), « Modifier » (`updateTreatment`), « Supprimer » (`deleteTreatment`).
- Un `TableView` (`tableTreatments`) avec cinq colonnes : ID, Médicament, Type, Posologie, Date Fin.
- Un bouton « Générer l'Ordonnance (PDF) » en bas à droite (`downloadPDF`).

Feuille de style loginDesign.css

La feuille de style CSS centralise l'ensemble des règles de mise en forme de l'application. Elle définit :

- Les couleurs de fond, de texte et les bordures.
- Les polices, tailles et espacements.
- Les styles des boutons (normal, hover, pressé).
- Les classes `.side-form`, `.bar`, `.shadow`, `.nav`, `.btn-deco`, `.card-info`, `.card-value`.

6.5 Fonctionnalités spécifiques

Filtrage dynamique avec `FilteredList`

La recherche en temps réel est implémentée dans `PatientController` et `TreatmentController` :

Listing 6.9 – Filtrage dynamique

```
1 filteredData = new FilteredList<>(patientList, p -> true);
```

```

2 txtRecherche.textProperty().addListener((obs, old, newVal) -> {
3     filteredData.setPredicate(patient -> {
4         if (newVal == null || newVal.isEmpty()) return true;
5         return patient.getNom().toLowerCase().contains(newVal.
6             toLowerCase().trim());
7     });
8 tablePatients.setItems(filteredData);

```

Chronomètre en temps réel

MainController utilise une Timeline pour afficher la date et l'heure dynamiquement :

Listing 6.10 – Affichage de l'heure en temps réel

```

1 Timeline timeline = new Timeline(
2     new KeyFrame(Duration.seconds(1), e -> {
3         LocalDateTime now = LocalDateTime.now();
4         dateLabel.setText(now.format(DateTimeFormatter.ofPattern(
5             "dd/MM/yyyy"))));
6         dayLabel.setText(now.format(DateTimeFormatter.ofPattern("
7             EEEE"))));
8         timeLabel.setText(now.format(DateTimeFormatter.ofPattern(
9             "HH:mm:ss"))));
10    });
11 timeline.setCycleCount(Timeline.INDEFINITE);
12 timeline.play();

```

Génération d'ordonnance imprimable

L'application intègre un système d'impression d'ordonnances via `PrinterJob`, permettant de générer un document structuré avec :

- En-tête institutionnel avec le nom du médecin.
- Coordonnées du patient.
- Tableau des médicaments prescrits.
- Signature et cachet.

Blocage de sécurité par allergies

Le système vérifie automatiquement si le patient est allergique au médicament prescrit avant d'enregistrer le traitement, empêchant ainsi toute prescription dangereuse.

Coloration dynamique des traitements expirés

Les traitements dont la date de fin est dépassée sont automatiquement surlignés en rouge pour attirer l'attention du médecin.

Chapitre 7

Interfaces de l'application

7.1 Fenêtre principale

L'interface de l'application **CareDash** s'articule autour de deux phases distinctes : l'authentification et l'espace de travail.

Écran de connexion

L'écran de connexion présente une interface épurée centrée sur un formulaire d'authentification. L'utilisateur y retrouve le logo « **SUIVI DE TRAITEMENTS MÉDICAUX** » et le libellé « **Espace Connexion** ». Un champ de saisie pré-rempli (exemple : `elmaachichama@gmail.com`) permet de renseigner l'adresse email, tandis qu'un second champ masqué est dédié au mot de passe. Deux boutons d'action principaux sont disponibles : « **Se connecter** » pour accéder à l'application, et « **Créer un compte** » pour basculer vers le formulaire d'inscription.

Écran d'inscription

L'écran d'inscription reprend la même identité visuelle que la page de connexion. Il propose quatre champs de saisie : « **Nom complet** », « **Votre adresse Email** », « **Votre mot de passe** » et « **Confirmer le mot de passe** ». Un bouton « **S'inscrire** » valide la création du compte, tandis qu'un lien « **Retour à la connexion** » permet de revenir à l'écran d'authentification.

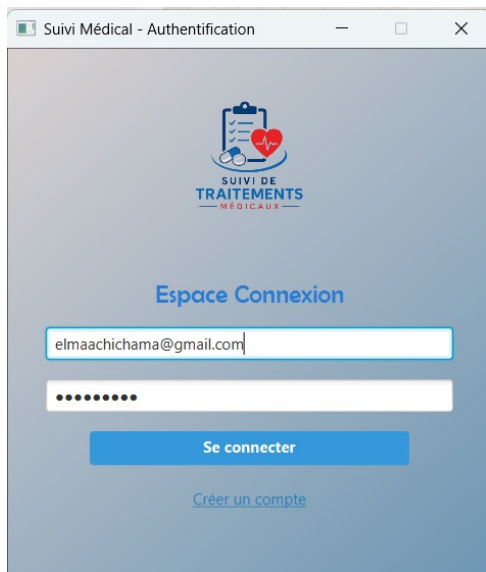


FIGURE 7.1.1 – Écran de connexion

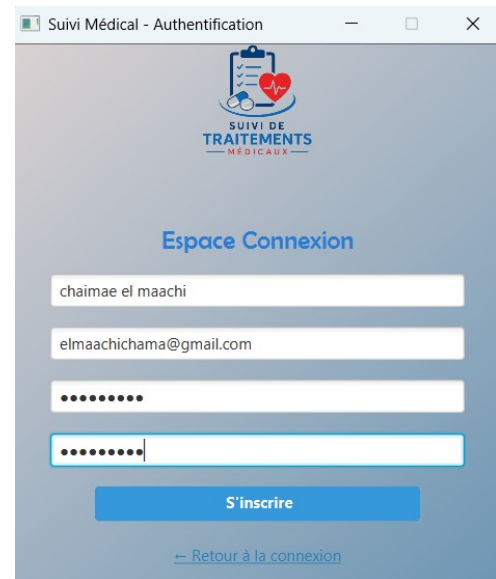


FIGURE 7.1.2 – Écran d'inscription

Espace de travail (Fenêtre principale)

Après authentification, l'utilisateur accède à la fenêtre principale (Figure 7.1.3). Celle-ci est divisée en deux parties distinctes :

- **Panneau de navigation latéral (gauche)** : il contient les trois modules principaux de l'application sous forme de liens cliquables : « **Tableau de Bord** », « **Patients** » et « **Ordonnances** ». Ce menu permet une navigation rapide entre les différentes fonctionnalités.
- **Zone de contenu principale (droite)** : l'en-tête affiche le titre « **Care Dash** », la date du jour (21/06/2026), le jour de la semaine (**dimanche**) ainsi que l'heure dynamique (16:02:32). Un message de bienvenue personnalisé s'affiche : « **Bonjour Dr.chaimae el maachi, Bienvenue dans votre espace de gestion et suivi medical. Veuillez selectionner une option dans le menu de gauche pour commencer.** ». Enfin, un bouton « **Deconnexion** » est positionné en bas du panneau latéral pour permettre une déconnexion rapide et sécurisée.

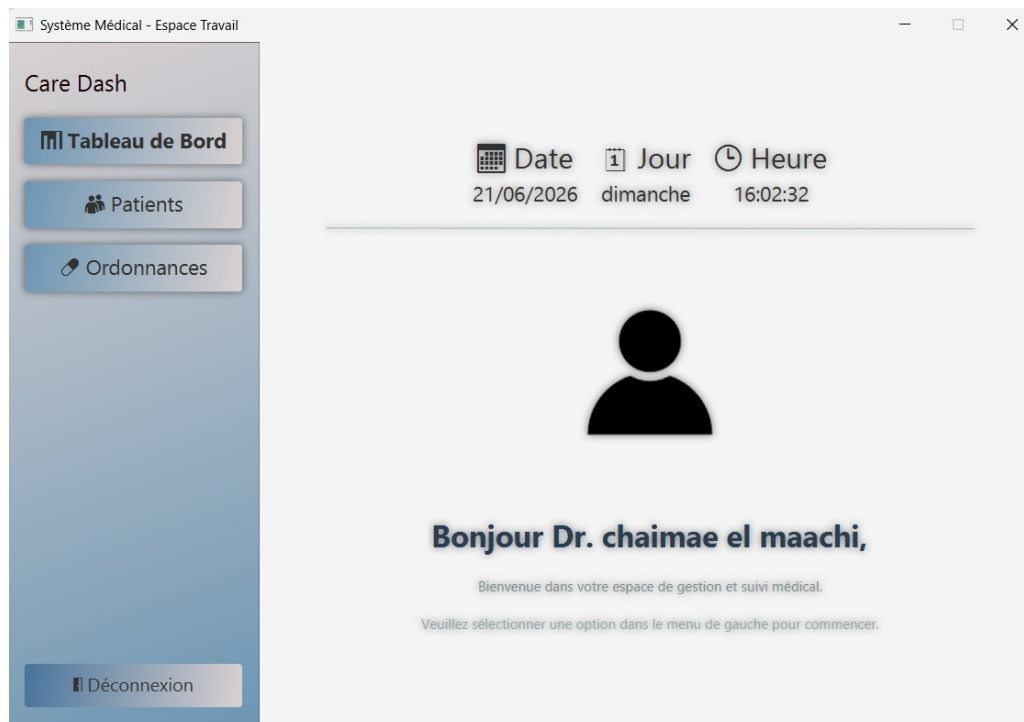


FIGURE 7.1.3 – Fenêtre principale de l’application (espace de travail)

7.2 Interfaces de gestion de l’entité 1 – Patients

L’interface de gestion des patients (Figure 7.2.1) permet d’assurer le cycle de vie complet des dossiers médicaux. Elle est organisée autour d’un tableau principal et de plusieurs contrôles interactifs :

- **Barre de recherche** : un champ intitulé « **Rechercher un patient** » permet de filtrer instantanément la liste des patients affichés. La recherche s’effectue en temps réel à mesure que l’utilisateur saisit du texte.
- **Champs de saisie et filtres** : quatre champs sont disposés au-dessus du tableau pour faciliter l’ajout ou le filtrage : « **Nom complet** », « **AAAA-MM-JJ** » (date de naissance), « **M/F** » (sexe) et « **Allergies (ex : Pénicilline)** ». Ces champs servent à la fois de critères de recherche et de formulaire de saisie pour les nouvelles entrées.
- **Boutons d’action CRUD** : trois boutons permettent de gérer les enregistrements : « **+ Aj** » (Ajouter), « **= Mod** » (Modifier) et « **Sup** » (Supprimer). Ces actions ouvrent des dialogues de confirmation ou de saisie pour éviter les manipulations accidentelles.
- **Tableau de visualisation** : intégré via un composant `TableView`, il affiche la liste complète des patients avec les colonnes suivantes : **ID**, **Nom**, **Naissance** (date), **Sexe** et **Allergies**. Les données présentées dans la capture incluent des exemples concrets : *amine HAH* (M, Pénicilline), *ihsan Imir* (F, Allergie alimentaire), *nour el madi* (F, Poils d’animaux) et *ahlam ziyadi* (F, Asthme allergique).

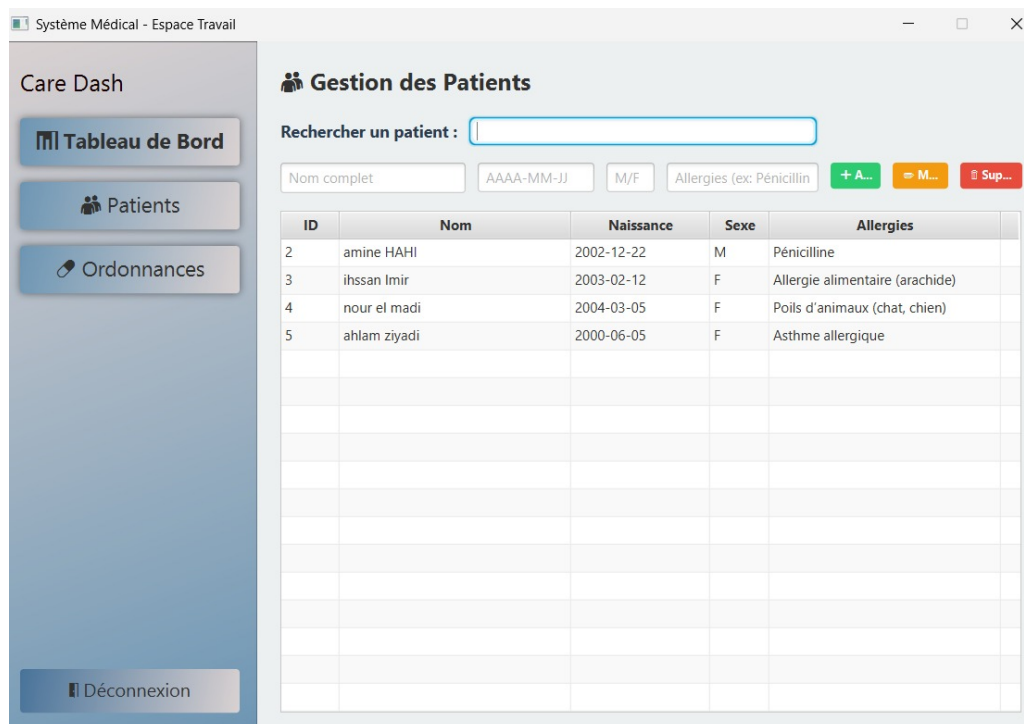


FIGURE 7.2.1 – Interface de gestion des patients

7.3 Interfaces de gestion de l'entité 2 – Ordonnances

L'interface de gestion des ordonnances (Figure 7.3.1) est dédiée à la prescription et au suivi des traitements médicamenteux. Elle se distingue par des fonctionnalités avancées de contextualisation :

- **Sélecteur de patient** : un menu déroulant intitulé « **Choisir un Patient** » permet d'associer l'ordonnance à un patient spécifique. Dans l'exemple, le patient sélectionné est 4 - **nour el madi**, ce qui filtre automatiquement la liste des traitements pour ne montrer que ceux qui le concernent.
- **Recherche contextuelle** : un champ de recherche avec l'instruction « **Tapez un médicament, type** » permet de filtrer les lignes d'ordonnances par médicament, type ou posologie, facilitant ainsi la recherche d'une prescription particulière.
- **Tableau des prescriptions** : il affiche les colonnes suivantes : **ID**, **Médicament**, **Type**, **Posologie** et **Date Fin**. La ligne présentée en exemple décrit une prescription d'*Antihistaminiques* de type *Cétirizine* avec une posologie de *10 mg une fois par jour* et une date de fin au 2026-06-24. Ce tableau permet au médecin d'avoir une vision claire et chronologique des traitements en cours ou terminés.
- **Génération d'ordonnance** : un bouton distinct « **Générer l'Ordonnance (PDF)** » est positionné en bas de l'interface. Cette fonctionnalité déclenche l'exportation du traitement sélectionné vers un document PDF officiel, prêt à être imprimé ou transmis au patient.



FIGURE 7.4.1 – Tableau de bord statistique

7.5 Menus et dialogues

L'application CareDash intègre plusieurs niveaux de navigation et de dialogues pour une expérience utilisateur fluide :

- **Menu latéral de navigation** : situé à gauche de la fenêtre, il constitue le principal système de routage. Il regroupe les trois accès majeurs : « **Tableau de Bord** », « **Patients** » et « **Ordonnances** ». La section active est généralement mise en surbrillance pour situer l'utilisateur dans son contexte.
- **Dialogues de saisie (Ajout / Modification)** : lors du clic sur les boutons « + Aj » (Ajouter) ou « = Mod » (Modifier) dans la gestion des patients, une fenêtre modale (dialogue) s'ouvre par-dessus l'interface principale. Elle reprend les champs (Nom, Date, Sexe, Allergies) et intègre une validation des données (ex : format de l'email, date cohérente, champs obligatoires) avant la soumission.
- **Dialogues de confirmation** : avant toute suppression définitive (bouton « Sup »), une boîte de dialogue de confirmation s'affiche pour éviter les manipulations accidentelles. L'utilisateur doit confirmer son intention en cliquant sur « Oui » avant que l'enregistrement ne soit définitivement effacé de la base de données.
- **Recherche et filtres en temps réel** : les champs de recherche (présents dans les modules Patients et Ordonnances) sont liés à des filtres dynamiques. Grâce à l'utilisation de `FilteredList` dans le contrôleur JavaFX, la liste se met à jour instantanément à chaque caractère saisi, sans recharger la page.

7.6 Export de données

La fonctionnalité d'export est principalement orientée vers la génération de documents médicaux officiels :

- **Génération d'ordonnance au format PDF** : via le bouton dédié « **Générer l'Ordonnance (PDF)** » présent dans l'interface de gestion des ordonnances, l'application construit un document PDF structuré. Ce document reprend les informations du patient sélectionné (nom, prénom), la liste des médicaments prescrits (avec posologie et durée), ainsi que les mentions légales nécessaires. Le PDF est formaté pour une impression sur papier A4.
- **Impression directe** : en complément de l'export PDF, l'application offre une prise en charge de l'impression directe via le gestionnaire d'impression de Java, permettant d'éditer physiquement les ordonnances pour les patients qui en font la demande.

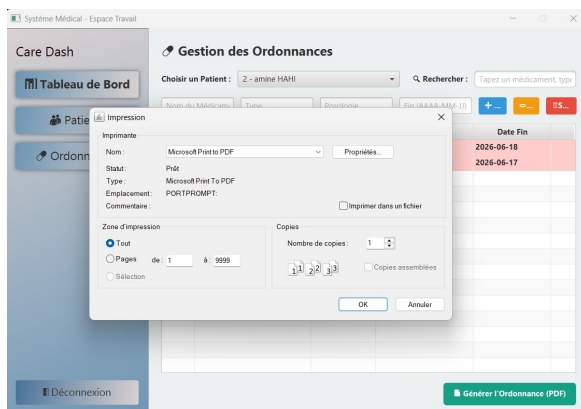


FIGURE 7.6.1 – Impression

CENTRE MÉDICAL INTERNATIONAL

Dr. chaimae elmaachi

Médecine Générale & Suivi Clinique Sévère
| Clinique

PATIENT : 2 - amine HAHI

ORDONNANCE MÉDICALE

Médicament	Type	Posologie & Durée
FF	G	URR (Fin: 2026-06-18)
FF	G	URR (Fin: 2026-06-17)

Cachet

FIGURE 7.6.2 – Fiche d'ordonnance

Conclusion

Le développement de **CareDash** nous a permis de mettre en pratique l'ensemble des concepts étudiés dans le module Développement Java IHM, en allant de la conception rigoureuse du design pattern MVC (Modèle-Vue-Contrôleur) à l'interaction JDBC directe et optimisée avec une base de données relationnelle. L'implémentation de fonctionnalités avancées, telles que le filtrage réactif en mémoire et la coloration dynamique conditionnelle, démontre qu'il est tout à fait possible de concevoir des interfaces riches, performantes et parfaitement adaptées aux exigences réelles du corps médical.

Au-delà de la simple validation des compétences techniques d'ingénierie logicielle, ce projet apporte une réponse concrète aux problématiques critiques de gestion clinique en éliminant les risques d'erreurs liés aux supports manuscrits et en fluidifiant le parcours administratif. L'accent mis sur l'ergonomie visuelle et la réduction des clics garantit une prise en main immédiate par le personnel soignant, optimisant ainsi leur temps de travail quotidien au profit du soin des patients.

En guise de perspectives, cette première version stable de CareDash pose des fondations solides pour de futures extensions. Il serait particulièrement envisageable d'y intégrer un module d'authentification multi-utilisateur hautement sécurisé, un système de sauvegarde automatique de la base de données décentralisée, ou encore des fonctionnalités d'exportation automatique des ordonnances et comptes-rendus au format PDF. Ainsi, le projet ne se limite pas à un exercice académique, mais s'inscrit dans une réelle démarche d'évolution et de transformation digitale de la gestion médicale.

Références

- Documentation officielle JavaFX : <https://openjfx.io>
- Oracle JavaFX Documentation : <https://docs.oracle.com/javafx/2/>
- JavaFX CSS Reference Guide : <https://docs.oracle.com/javafx/2/api/javafx/scene/doc-files/cssref.html>
- JavaFX Scene Builder User Guide : <https://docs.oracle.com/javafx/scenebuilder/1/>
- Documentation MySQL Connector/J : <https://dev.mysql.com/doc/connector-j/en/>
- Documentation MySQL : <https://dev.mysql.com/doc/>
- Supports de cours du module "Développement Java IHM" ENSA Oujda.

Ces ressources ont permis d'acquérir les connaissances nécessaires à la conception et au développement de l'application CareDash, aussi bien pour la partie interface utilisateur (JavaFX) que pour la gestion de la base de données (MySQL) et l'architecture logicielle (MVC).